Strata — Hub Guide

For hub operators

strata-kb v1.35.0 · 2026-11-05

1. Who this guide is for

You run the hub. It is the repository whose `federation/` folder holds the one copy of everything the organisation can search, and it serves that copy to people, agents and CI. You decide who may publish, you merge what they send, and you keep the service running.

> **Nothing a child, BA or Dev repo publishes is searchable until a pull request on this hub merges** (and the running hub pulls it). You are that gate, unless you switch auto-merge on for a repo (section 4.3).

A hub does	A hub does not
Hold `federation/`, the published snapshot of every repo, and search only that	Search its own working `.kb/`: until the hub publishes its own documents, the server cannot find them
Serve the HTTP MCP server, the REST API and the Web UI, and accept publishes from CI (intake)	Let an unregistered repo publish once `federation/registry.yaml` is non-empty
Review and merge publish PRs; keep accounts, the registry, Ticket Q&A and ticket claims	Write child, BA or Dev content for them: those repos author and publish it themselves

A hub repo can author documents with exactly the same steps as a child repo. The steps are in the child guide; this guide does not repeat them (section 3.5 covers what differs on a hub). BA repos and Dev repos publish to the same hub and have their own guides: BA and Developer.

Shared concepts, such as the L0 to L3 layers and `kb-context` citations, are explained in the architecture document.

Slash commands. Every slash command in this guide exists for Claude Code, GitHub Copilot and Cursor, with these exceptions:

- `/kb-summarize` has no Copilot prompt. Copilot loads `.github/instructions/kb-summarize.instructions.md` while you work in `.kb/**`. Cursor also has a rule, `.cursor/rules/kb-summarize.mdc`.
- `/kb-ingest` and `/kb-publish` are skills in Claude Code (`.claude/skills/<name>/`), not files in `.claude/commands/`. You still type the slash command.
- `/kb-l3-fix` is useful only while the L3 AI fix is switched on (child guide).
- A hub has no `/kb-mcp-setup` and no `/kb-auto`.

The commands a hub gets: `/kb-init`, `/kb-docker-setup`, `/kb-ingest`, `/kb-summarize`, `/kb-l3-fix`, `/kb-approve`, `/kb-publish`, `/kb-code-sync`.

2. First-time setup

2.1 Install and scaffold

```
pip install strata-kb
mkdir my-hub && cd my-hub && git init
kb init --kind hub
```

You need Python 3.11 or newer, Git and, for the service, Docker. Add the `ingest` extra (`pip install "strata-kb[ingest]"`) only if you also author documents on this machine. Commit the result and push it to a Git host: the hub is an ordinary Git repository, and publishers open their PRs against it.

`kb init --kind hub` writes `.kb/config.yaml` (`kind: hub`, `hub: "."` so the repo is its own hub, `repo_id` from the folder name), `.kb/index.yaml`, `docker-compose.yml` and `.env.example`, the CI workflow `.github/workflows/kb-publish.yml`, `.gitattributes` (LF line endings), and `.gitignore` (it keeps `.kb-work/`, `.kb-auth/`, `.kb-qa/` out of Git; `source/` and `sources/` have their own, for the PDFs), `QUICKSTART.md` and the eight skills and commands. Its `.mcp.json` and `.cursor/mcp.json` run a **stdio** MCP server for you (`python -m strata_kb.mcp --kb .kb/`), not the shared one. Re-running `kb init` is safe: it creates missing files and refreshes scaffold files, but skips `.kb/config.yaml` and `.kb/index.yaml` unless you pass `--force`. Pass `--assets s3` to choose S3 asset storage (2.7).

2.2 Start the service

```
kb docker-setup        # in your assistant: /kb-docker-setup
```

It writes `.env` with a fresh `STRATA_KB_HTTP_TOKEN` (48 hex characters, never printed), adds `.env` to `.gitignore` and runs `docker compose up -d` when Docker is ready. `--no-docker` only prepares the files and prints the next steps. If `.env` already exists the command stops, unless you pass `--force` (on a terminal it asks) to regenerate the token and keep every other line. Without the helper: `cp .env.example .env`, put your own token in it, `docker compose up -d`.

The compose file runs two services: `hub` (port 8321, `./` mounted at `/data`, a named volume for the model cache, health check `/api/health`) and `hub-sync` (the daily code-sync scan, section 4.4). The image tag is the version of the `kb` that scaffolded it. To run without Docker:

```
export STRATA_KB_HTTP_TOKEN=$(python -c "import secrets; print(secrets.token_hex(24))")
python -m strata_kb.mcp --hub . --transport http --host 0.0.0.0
```

`--host` defaults to `127.0.0.1` and `--port` to `8321`. Add the `embed` extra (`pip install "strata-kb[embed]"`) for the semantic leg of search. A systemd unit is in `docs/deploy-remote-mcp.md`. One process serves four surfaces:

Surface	For	Auth
`/mcp`	AI agents	Bearer token
`/api/…`	scripts and integrations	Bearer token, or the session cookie
`/ui`	people	a hub account (2.4); the bearer token opens the UI read-only
`/intake/…`	CI publishes	GitHub Actions OIDC; on only when 2.5 is done

The hub is mandatory: the server takes it from `--hub`, then `STRATA_KB_HUB`, then `.kb/config.yaml`, and refuses to start without one.

The token. The HTTP server refuses to start without `STRATA_KB_HTTP_TOKEN`, and refuses the `change-me` placeholder, a token shorter than 32 characters, or one that is not printable ASCII without spaces (the documents are copyrighted: never run HTTP without authentication). The token serves `/api`, `/mcp` and CI; people sign in to `/ui` with an account instead.

2.3 Before you expose it

- **Replace the generated token.** `kb docker-setup` generates one for convenience. Treat it as a placeholder: put your own secret in `.env` and keep it in a secret manager. Every repo that connects over MCP holds this token. After any change to `.env`, recreate the service: `docker compose up -d --force-recreate` (all services: `hub-sync` reads the same `.env`).
- **Behind a reverse proxy, set `STRATA_KB_TRUSTED_PROXIES`** to the number of trusted proxies in front of the server (an integer, default `0`; anything else stops startup). Left at `0` behind a proxy, every caller looks like the proxy: one caller can exhaust the rate limiters for everyone, and one locked-out user locks out every login. Set, it also lets the `/ui` session cookie earn its `Secure` flag from `X-Forwarded-Proto`; unset, the server logs a warning and sets the cookie without `Secure`. Each proxy must append to `X-Forwarded-For` and `X-Forwarded-Proto`, not overwrite them. At `1` or more, bind the server so that only your proxy can reach it: a client that connects directly controls its own rate-limit key.
- **Set `STRATA_KB_ALLOWED_HOSTS`** to the public host name(s) the proxy serves, comma separated, bare names only (`kb.example.com`; a scheme, port or path stops startup). Unset, `/mcp` behind a proxy at a real name answers `421 Invalid Host header` while `/api` and `/ui` work.
- **Know the limit.** HTTP auth is a bearer token for `/api`, `/mcp` and CI, plus local hub accounts for the UI: no OAuth or SSO. That is adequate for an internal network and not ready for the public internet. TLS belongs at the proxy, and so does HSTS: behind a proxy the server never sees https, so it sends none.

The full deployment notes are in `docs/deploy-remote-mcp.md`.

2.4 Create the first admin

People sign in to `/ui` with a username and password. The accounts live in `<hub>/.kb-auth/` (`users.db` and `session.key`). **Never commit that directory**: a new hub's `.gitignore` lists it and the running hub also adds it to `.git/info/exclude`. Deleting `session.key` logs everyone out. Pick one way to create the first account:

- **Setup page (default).** Open the hub in a browser. While no account exists every `/ui` page redirects to `/ui/setup`. Read the one-time setup code from the hub log (`docker compose logs hub`; with systemd, `journalctl -u <unit>`), enter it with a username and password, and that account becomes the `admin`. The code lives only in memory and changes on every restart; the bearer token never opens this page. Once an account exists, `/ui/setup` returns 404.
- **Command.** `docker compose exec hub kb user add <name> --role admin` (it prompts for the password twice).
- **Bootstrap variables.** In `.env`, set `STRATA_KB_BOOTSTRAP_ADMIN=<name>` and `STRATA_KB_BOOTSTRAP_PASSWORD_HASH='<hash>'`, where `<hash>` is the output of `kb user hash` (it prompts for the password; run it on any machine with strata-kb). **Keep the single quotes**: Docker Compose expands `$` in `.env`, and the hash contains several. Plaintext is never accepted. Once any account exists both variables are ignored, and no setup code is printed.

Roles are `admin`, `ba` and `dev`; only `admin` sees and uses the Registry page. More accounts: section 3.2.

2.5 Register the publishers

The intake accepts a publish only from a repository listed in `federation/registry.yaml`, and that entry (never the payload) decides which `repo_id` the repo may write. Add every publisher with a normal PR:

```
repos:
  acme/handbook: handbook                              # plain form: just the repo-id
  acme/safety-docs: {repo_id: safety, kind: kb}        # a child: Document menu
  acme/payments: {repo_id: payments, kind: dev}        # a dev repo: -code and -svc, Code menu
  acme/requirements:                                   # a ba repo: -plan, Ticket menu
    repo_id: acme-requirements
    kind: ba
    auto_merge: true                                   # optional: section 4.3
```

- **The hub becomes governed.** Once the file is non-empty, `kb publish` over git refuses any repo-id not listed here and refuses direct pushes. Every repo that publishes over git, not only those that use the OIDC intake, must be registered. The hub's own self-publish (3.5) is exempt.
- **`repo-id` must exactly match `repo_id:`** in that repo's `.kb/config.yaml`. Over git a mismatch is refused before anything is written, naming both ids. Over the intake the publish still succeeds and the PR opens, but the publisher's `kb publish` times out waiting for a PR it cannot find, and its uploads lose incrementality (every publish becomes a full upload). A `ba` repo with `repo_id: acme-requirements` publishes `acme-requirements-plan`.
- **`kind`** is `kb`, `ba` or `dev` and decides the hub menu (Document, Ticket, Code). The registry wins: a publish whose repo kind maps to something else is refused (`kb publish` exits with `kind mismatch`; the intake answers `403` with the same text). An entry without `kind` still works and the repo's own kind is recorded. On an existing hub, `kb init --kind hub` proposes a kind for each kind-less entry, shows the diff and asks (`--yes` writes without asking). It never commits: open the PR yourself, and only after the publishers that read the registry themselves (direct `kb publish`, child hubs) run the hub's version.
- **Leave `workflow:` unset.** The intake compares it with the full `job_workflow_ref`, which ends in `@refs/tags/kb-publish/<timestamp>`, so a pin never matches and every publish is refused. An entry accepts only `repo_id`, `kind`, `workflow` and `auto_merge`; any other key makes the registry invalid.
- **Branch protection on the hub plus the OIDC intake are the actual security boundary.** The registry cannot authenticate a git publisher (a remote URL is self-asserted), so protect the default branch: that is what makes the review route binding.

Turn the intake on. Create a GitHub App (hub owner) with `Contents: Read and write` and `Pull requests: Read and write`, install it on the hub repo only, and download the private key. Then set, on the server:

Variable	Value
`STRATA_KB_GH_APP_ID`	the App id
`STRATA_KB_GH_APP_KEY`	the **path** to the PEM file, readable by the server
`STRATA_KB_INTAKE_AUDIENCE`	the URL your repos put in `intake:`, without a trailing slash

All three present turns `/intake/*` on. Any missing, or an unreadable PEM, leaves intake off (the log says `intake disabled`), while read-only MCP, API and UI keep working. In Docker the PEM must exist inside the container, and the scaffolded compose mounts only `./:/data`: mount the key read-only in a `docker-compose.override.yml` (`kb init` refreshes `docker-compose.yml`), and never leave it in a tracked path. The same App pushes Registry edits (3.3) and merges PRs under auto-merge (4.3).

2.6 Give repo owners their settings

Each repo owner needs the following from you, and a registry entry from 2.5:



`hub:` (a git URL or path) and `STRATA_KB_HUB_URL` (the hub's HTTP base URL, used by MCP) are different values.

One version for every repo. Every repo in the federation runs the hub's strata-kb version. Upgrade the hub first (3.8), then tell each repo owner to run `kb doctor` in their child, BA or Dev repo. Once their MCP is set up (`kb mcp-setup`), `kb doctor` asks the hub for its version, installs it, sets the `strata-kb==<hub version>` pins in `.github/workflows/`, and refreshes their skills and commands. In CI it installs nothing and reports the mismatch. The pin matters: an unpinned `pip install strata-kb` in a job that holds `id-token: write` would let every run pick up whatever PyPI serves, in a job able to write to the hub.

To rotate the token, set the new one in `.env`, recreate the services (2.3), and tell the owners to run `STRATA_KB_HTTP_TOKEN=<new> kb mcp-setup`.

2.7 Optional: asset storage in S3

Image assets live in Git (`mode: none`, the default) or in an S3-compatible bucket. Only the hub configures this. `kb doctor` warns once the assets in Git pass about 100 MB.

1. Create a **private** bucket (block public access) and credentials scoped to Get, Put and Head on the `assets/` prefix only. Export them through the standard AWS environment chain (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, region); they never go in `config.yaml`. For MinIO, R2 or another S3-compatible host, set `endpoint` instead.
2. `pip install "strata-kb[s3]"` on every machine that publishes or serves.
3. `kb init --kind hub --assets s3`, or edit the block by hand:

```
# .kb/config.yaml
asset_store: {mode: s3, bucket: my-kb-assets, region: eu-west-1, prefix: assets/}
```

4. `kb assets migrate` moves the assets already in Git into the bucket, one repo-id at a time, and commits locally; it never pushes, so push it like any hub commit. `kb assets verify` checks that every record resolves, every markdown reference is covered and nothing is orphaned. `kb doctor` confirms the store is reachable.

From then on a publish uploads images to the bucket instead of committing them, and `/assets/<name>` serves them with a local disk cache. To go back, download each asset named in the `_assets.yaml` files, delete those files, set `mode: none` and commit.

2.8 Optional: chain hubs (multi-tier)

A `kind: hub` repo may itself declare `hub:` (the upstream hub) and `repo_id:`. Publishing it mirrors its **`federation/`** (not its own `.kb/`) into `federation/<hub-id>/` upstream, keeping the nested layout; ids become paths (`mid/repo-x:doc-id`), depth is unbounded. Choose which hub people query: a team hub returns the team's knowledge, the root hub everything.

- **Only `kb` goes upward.** A child hub shows all three menus and accepts `ba` and `dev` publishes, but sends its parent only its own `kb` leaves. `kb publish` prints one `skipped <leaf> (kind <kind> stays on this hub)` line for each leaf held back. A parent never serves a non-`kb` leaf nested under a child hub, and `kb doctor` names it.
- Hub-to-hub publishing needs direct Git access upstream, or `gh` for `--pr`; the intake route is for child, BA and Dev repos only. A chain that would loop is refused: `federation cycle detected`. An emptied local `federation/` does not propagate deletions upstream (publish refuses rather than wipe entries); removing some entries does, as an ordinary mirror in a PR on the upstream hub.

---

3. Daily work

Task	Command	Result
Review a publish PR	on your Git host; see 3.1	merging it is the gate
Add, change or remove an account	`kb user` (`add`, `passwd`, `role`, `remove`, `list`)	`.kb-auth/users.db`
Register a repo, switch auto-merge	Registry page (`/ui/registry`), or a PR on `federation/registry.yaml`	the repo may publish
Release a stuck claim, read Q&A	Ticket page, `/ui/questions`	3.4
Author on the hub	the child guide steps, then `kb publish`	documents in `federation/<repo-id>/`
Search	`kb query`, `kb get`, `kb tags`, the Web UI	cited sections
Check health	`kb doctor`	`[error]`, `[warning]`, `[info]` lines
Repair the index	`kb reindex`	`federation/index.yaml` rebuilt
Upgrade	3.8	every repo on one version

3.1 Review and merge publish PRs

Each publish arrives as a PR on this hub that changes `federation/<repo-id>/` and the aggregate `federation/index.yaml`. Merging it is the gate: after the merge the hub pulls the new commit (4.1) and the content is searchable. What each kind of repo sends, and what was checked before it reached you:

From	You get	Already checked
Child (and your own hub)	documents: all four layers (L0 to L3)	no unconfirmed L3 finding (publish refuses otherwise); unreviewed sections only warn, unless the publisher used `--require-reviewed`
BA repo	`<repo-id>-plan`: its missions and its **ready** tickets	the Definition of Ready (`kb plan-ingest` skips the rest)
Dev repo	`<repo-id>-code` (generated by `kb code-ingest`) and `<repo-id>-svc` (curated)	`-svc` is reviewed and approved in the Dev's own repo before publishing, so a `-svc` amend riding along is not re-reviewed here

For a document PR, read the diff like a document with tracked changes. Check the new prose (the `.md` files, not `.raw.md`) against your understanding of the subject: nothing important omitted, nothing present that the source does not say. Check that every code, identifier, number and unit is exactly as in the source. Do not verify tables line by line, because `kb build` already compared them; glance only for extraction errors (a row or column the PDF reader misread), which a machine cannot catch. Check that the one-line summaries in `_manifest.yaml` say what each section is about, well enough that search will find it. If something is wrong, edit the `.md` or `.yaml` in your Git host's web editor, comment why, or ask the publisher to fix it.

A PR that was closed or merged and whose `publish/<repo-id>` branch was deleted is fine: the next publish opens a fresh one.

3.2 Manage accounts

```
docker compose exec hub kb user add <name> --role ba   # prompts for the password twice
docker compose exec hub kb user passwd <name>          # also: role <name> <role>, remove <name>
docker compose exec hub kb user list                   # never prints hashes
```

Roles are `admin`, `ba` and `dev`. The last admin cannot be removed or demoted. Deleting `.kb-auth/session.key` logs everyone out. Outside Docker, run the same commands with `--kb <hub>/.kb`. The bearer token is not an account: in the UI it is read-only and never opens the Registry page.

3.3 Registry and auto-merge

The Registry page (`/ui/registry`, `admin` only) adds or removes a repo and switches its **auto-merge** on or off. Each edit is a commit on the hub's default branch, pushed with the intake GitHub App (message `registry: <action> <repo> (by <user>)`) and never force-pushed. If the default branch is protected, the App must be allowed to push to it, or every edit is rejected. The page is read-only, with a banner saying why, when intake is not configured or the hub has no Git remote. Removing a repo does not delete its published content. Editing `federation/registry.yaml` by PR (2.5) does the same thing and needs no App.

3.4 Questions and claims

Ticket Q&A. A Dev who meets a business question about a published ticket asks the BA through the skills; the BA answers; the hub keeps the thread. You configure nothing: the store is created on the first Q&A call (6.5). The read-only **Q&A** menu (`/ui/questions`) lists the tickets that have questions, grouped by BA repo, with open, waiting and answered counts (`?all=1` includes fully answered tickets), and `/ui/questions/ticket?repo=<repo-id>&ticket=<id>` shows one ticket's thread.

Claims. A Dev's `kb claim take` keeps two Devs off the same ticket. A claim expires after `STRATA_KB_CLAIM_TTL` seconds without an update (default `14400`, 4 hours); a claim in `review` or `publishing` with a PR, and a `stopped` one, never expires. A hub admin releases anyone's claim with the **Release** button, and a reason, on the ticket page of the Ticket menu.

3.5 Author on the hub itself

A hub ingests, summarizes, builds and approves documents with the same commands as a child: follow the child guide. What differs:

- `hub:` is `.`, so `kb publish` is a **self-publish**: it mirrors `.kb/` into `federation/<repo-id>/` on this very repo. Until then the server cannot search the document. Self-publish is exempt from the registry. With a Git remote it opens a PR (it needs a working `gh`); with no remote it commits directly.
- `kb ingest` also runs in the container: `docker compose run --rm hub kb ingest …`.
- A document whose Markdown is now the master is tagged with `kb authored <doc>` (`--off` removes the tag); `kb ingest` then refuses to re-ingest it, so a re-ingest cannot overwrite the hand fixes. `kb summarize --redo` stays allowed.
- Source PDFs stay in `source/`, which Git ignores. Do not commit them.

3.6 Search and the Web UI

```
kb query "parental leave eligibility"
kb query "parental leave" --tags hr --budget 600 --kind kb
kb get hr-handbook 4.12 --level l2     # or --level l3
kb tags                                # every tag published on the hub
```

```
kb stats                                # token size per layer, per document
```

On a hub these read `federation/` only. Every result cites `<repo-id>:<doc-id> §<section> (<revision>)`. Query in the document's own language: the store never translates. `--semantic` forces the semantic leg and warns when the `embed` extra is missing.

The Web UI left rail is **Overview, Document, Ticket, Code, Q&A**: Document shows `kb` leaves, Ticket the BA plans (a status per story), Code `-code` and `-svc` at `/ui/code`. The top search box searches all kinds in one list.

3.7 Check health

```
kb doctor      # exit 1 on an error, 2 when the hub cache or a --context citation is stale
kb reindex     # rebuild federation/index.yaml when it has drifted; --force also rebuilds search
```

`kb doctor` reports broken contents and missing files, published content modified in place on the hub, files a publish would never have written, a hub clone storing a credential in its own `.git/config`, and a Git older than 2.31. On a hub it also checks kinds: registry entries without `kind` (a warning; fix with `kb init --kind hub`), a leaf whose recorded kind clashes with the registry, and non-`kb` leaves under a child hub. `federation/index.yaml` records `content_sha256` per snapshot, so tampering is detectable. Run `kb doctor` after an upgrade: the checks get stricter, and each message names the command that fixes it.

3.8 Upgrade the hub

Upgrade the hub first; repos follow (2.6).

```
pip install --upgrade strata-kb
kb init --kind hub        # refreshes the compose image tag, skills and workflows
docker compose pull && docker compose up -d
kb doctor
```

Commit what `kb init` changed. Then tell the owners of every child, BA and Dev repo to run `kb doctor`. `kb doctor` does not sync the hub repo itself: `kb init` is how it moves.

---

4. Automation modes

A hub has no `kb auto`. What runs without you:

Mode	What it does	Switch
Auto-pull	the `hub` service pulls the Git checkout so merged PRs go live	`STRATA_KB_HUB_PULL_INTERVAL`
CI publish via OIDC	a repo's own CI publishes to this hub with no stored secret	the three intake variables (2.5)
Auto-merge	the intake merges a repo's publish PR itself	per repo: Registry page, or `auto_merge: true`
Code-sync	the `hub-sync` service flags hub documents that name code that changed	`kb code-sync`, `/kb-code-sync`

4.1 Auto-pull

The HTTP server runs `git pull --ff-only` on the hub checkout every `STRATA_KB_HUB_PULL_INTERVAL` seconds (default `300`; `0` turns it off) and re-syncs the search index when HEAD moved, so a merged PR is searchable without a restart. A hand-edited or diverged checkout is never touched: the pull fails, a warning is logged, and the server keeps serving what it has. Make your own edits in another clone.

4.2 CI publish via OIDC

`kb ci-publish` is the CI route: it runs in the publisher's own GitHub Actions job, authenticates to this hub's intake with GitHub Actions OIDC, and needs no secrets.

1. A person runs `kb publish` in the repo, which tags the committed content `kb-publish/<timestamp>` and pushes the tag.
2. The tag starts the workflow, which runs `kb ci-publish` with `id-token: write`.
3. The intake verifies the token, checks the registry, and opens the PR on this hub (and merges it, if auto-merge is on for the repo). The publisher's command polls for the PR link and times out after 600 seconds.

The intake accepts only `refs/tags/kb-publish/*`. Which events create such a tag:

Repo	What publishes
Child	`kb publish` only. Merging to the default branch publishes nothing
Dev	`kb publish` only (its `kb-code.yml` runs on the tag; merging publishes nothing)
BA	`kb publish`, **or a merge to `main` that touches `missions/` or `tickets/`**: the repo's `kb-publish-request.yml` tags the merged commit and dispatches `kb-publish.yml` on that tag (`main` as scaffolded; another default branch means editing `main` in both places of that workflow)

With Registry auto-merge on for a BA repo, a BA merge therefore reaches this hub with no review on the hub side: apply your review rules on that repo's `main`. Leave `workflow:` unset for all of them (2.5).

4.3 Auto-merge per repo

Right after the intake opens a repo's publish PR, it merges it (a merge commit, or a squash when the repo forbids merge commits). It merges the **whole** PR, including any `-svc` content riding along. The App needs `Contents` and `Pull requests` write (2.5). If the merge fails, the PR stays open for a human and the publish still counts as done (`merged: false` with the reason in `detail`). `kb publish` and `kb ci-publish` print "merged" only when it merged; the CI step summary still shows `hub PR: <url>`. Turn it on only for repos whose content you do not need to review here.

4.4 Code-sync

A hub document written by hand or ingested from a PDF (architecture, standards, designs) goes stale when the `-code` or `-svc` documents it names change. Code-sync finds those documents and keeps them true, with no model in the daily job.

1. **Mark the documents.** `kb authored <doc>` on each one whose Markdown is now the master. A document tagged `code-sync-skip` is left out of the scan. `kb code-sync scan` prints a `note:` for each hub document that

is not `authored`.

2. **Set the baseline once.** `kb code-sync baseline` records the hub commit code-sync is current at, in `kb-code-sync.yaml` at the hub root. Commit it to the default branch. `--at <sha>` records another commit.
3. **Daily detection.** The `hub-sync` service runs `kb code-sync run --issue --loop`: scan, update the issue, sleep `STRATA_KB_CODE_SYNC_INTERVAL` seconds (default `86400`, minimum `60`). It never pulls, commits or pushes (the `hub` service pulls `/data`). It compares the `-code` and `-svc` documents between the baseline and the tip of the default branch and lists hub sections that name something added, removed or changed. One issue labelled `kb-code-sync` stays open while there are findings: rewritten only when they change, closed with a comment when the scan finds nothing. A scan error never closes it.
4. **Sprint cadence.** The hub manager runs `/kb-code-sync` in their agent any time the issue is open. The skill re-runs the scan, judges each finding (fix, no change needed, or needs the SA), edits the sections, re-summarizes, runs `kb build --strict`, moves the baseline, and publishes with `kb publish --branch kb-bot/code-sync`: one rolling PR that later runs add to. The SA or hub manager reviews it at sprint review, may run `kb approve`, and merges. The skill never merges and never approves.

Findings fall into `removed` (code gone, prose likely wrong), `changed`, `added` and `image`, in that order. `kb code-sync issue` judges the open issues (`current`, `stale`, `duplicate`, `foreign`); the skill works only from a usable one and never edits, closes or comments on an issue.

Prerequisite	Why
`origin/HEAD` set (`git remote set-head origin --auto`)	the scan reads the default branch's tip; a hub with a remote and no `origin/HEAD` exits `1`
A GitHub App with `issues: write`, `pull_requests: read`, `contents: read`	`hub-sync` calls the issues REST API and reads the baseline from the hub repo. The intake App needs these added and the installation re-approved by an org admin
or `STRATA_KB_GH_TOKEN`	a token with the same three permissions, used only when the App is not configured
`gh`, logged in, on the manager's machine	`kb publish --branch` and the PR title and body

With no credentials, `hub-sync` still scans and logs the work order, logs one line that the issue was not updated and why, and keeps running.

---

5. Troubleshooting

Symptom	Fix
Server exits `STRATA_KB_HTTP_TOKEN is the placeholder, … characters; at least 32 are required, or must be printable ASCII`	`kb docker-setup --force`, or set a token of 32 or more printable ASCII characters, no spaces (2.2)
Server exits on `STRATA_KB_ALLOWED_HOSTS entry … is not a host name`, or on `STRATA_KB_TRUSTED_PROXIES`	Bare host names only, comma separated; an integer of 0 or more (2.3)
`/mcp` answers `421 Invalid Host header`, `/api` and `/ui` work	Set `STRATA_KB_ALLOWED_HOSTS` to the public host name (2.3)
One failed login locks out everyone, or one caller exhausts the rate limits	The proxy hides real callers: set `STRATA_KB_TRUSTED_PROXIES` (2.3)
Every `/ui` page redirects to `/ui/setup`	No account exists yet (2.4). After the first account, `/ui/setup` is a 404
Registry page is read-only, with a banner	Intake is not configured (2.5), or the hub checkout has no Git remote `origin`
Registry edits are rejected	The default branch is protected and the App may not push: allow it
Log says `intake disabled /intake/*` is missing	One of the three variables is missing, or the PEM at `STRATA_KB_GH_APP_KEY` is unreadable. In Docker the path must exist inside the container
Publisher: `is not registered`, HTTP 403	Add `owner/repo` under `repos:` in `federation/registry.yaml` (2.5)
Publisher: `pinned to workflow … in federation/registry.yaml`	Remove the `workflow:` pin
`kind mismatch: registry says …`	Fix `kind:` in the registry entry or in the repo's `.kb/config.yaml`; the registry's kind wins
A publisher's `kb publish` times out but the PR exists	`repo-id` in the registry differs from the repo's `repo_id:`; fix one (2.5)
Publisher over git: direct push refused, or repo-id not listed	The hub is governed: register the repo, and publish by PR
Upload refused with `400`, path too long	Paths are capped at 110 UTF-16 code units so that `federation/<repo-id>/<path>` fits a Windows client; shorten the document or file name
Merged on the publisher's repo, not searchable	It is searchable once the **hub** PR merges and the hub pulls (up to `STRATA_KB_HUB_PULL_INTERVAL` seconds). Check the server log for a failed pull: a hand-edited or diverged checkout is left alone
Content arrived with no hub review	Auto-merge is on for that repo (4.2), possibly after a BA merge to `main` (as scaffolded, 4.2)
`kb doctor` names files inside a federation entry	An older publish mirrored more than `.kb/` artefacts. Publish again to strip them, and **rotate any credential they contained**: removing a file does not remove it from Git history
On a hub document, `kb summarize` skips it (`[skip] <doc>: L3 review pending`) and `kb build` or `kb publish` refuses: `L3 review pending`	An L3 finding is unconfirmed: `kb l3 report <doc-id>`, check each section against the PDF, then `kb l3 confirm <doc-id> --section <id>` (or `--all`); see the chat guide
Every hub command fails with an authentication error	`git --version`: below 2.31 with a credentialed hub URL, nothing works. Upgrade Git
`Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`	The client's MCP is a Dev repo's stdio server or a cache clone of the hub URL; point it at this server's `/mcp` (5.5)
`hub-sync` logs that the issue was not updated	No App credentials or `STRATA_KB_GH_TOKEN`, or missing permissions (4.4)

6. Appendix

6.1 Environment variables

Variable	Meaning
`STRATA_KB_HTTP_TOKEN`	the server's bearer token; required for `--transport http`; 32 or more printable ASCII characters, not `change-me`
`STRATA_KB_HUB`	overrides `hub:`; `--hub` wins over it. Carries a credentialed URL, which never goes in `hub:`
`STRATA_KB_ALLOWED_HOSTS`	bare host names `/mcp` accepts besides localhost, comma separated
`STRATA_KB_TRUSTED_PROXIES`	number of trusted reverse proxies, an integer of 0 or more, default `0`
`STRATA_KB_HUB_PULL_INTERVAL`	seconds between automatic pulls, default `300`, `0` = off
`STRATA_KB_HUB_CACHE`, `STRATA_KB_HUB_TTL`	the clone cache a command uses to read a hub URL (default `~/.strata-kb/hub/`) and how long it counts as fresh (default 900 seconds)
`STRATA_KB_GH_APP_ID`, `STRATA_KB_GH_APP_KEY`, `STRATA_KB_INTAKE_AUDIENCE`	the intake: App id, path to the PEM, expected OIDC audience; all three or none
`STRATA_KB_GH_TOKEN`	code-sync issue token when no App is configured
`STRATA_KB_CODE_SYNC_INTERVAL`	seconds between `hub-sync` scans, default `86400`, minimum `60`
`STRATA_KB_BOOTSTRAP_ADMIN`, `STRATA_KB_BOOTSTRAP_PASSWORD_HASH`	the first admin while no account exists; the hash is the output of `kb user hash`; plaintext is never accepted
`STRATA_KB_CLAIM_TTL`	seconds a ticket claim lives without an update, default `14400`; a non-positive value warns and falls back
`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, …	S3 credentials, from the standard AWS chain; never in `config.yaml`

6.2 Config keys, the registry and storage

A hub reads these from `.kb/config.yaml`: `kind: hub`, `hub` (`.` for a root hub; a different path or URL chains it upstream, 2.8), `repo_id`, `asset_store` (`mode`, `bucket`, `region`, `endpoint`, `prefix`; the default prefix is `assets/`), `l3_fix.enabled` (child guide), and `intake` (refused for a hub-to-hub publish, allowed for a self-publish). `.kb/index.yaml` `llm:` holds `runner`, `model`, `effort`, `max_workers` and `timeout`. At the hub root, `kb-code-sync.yaml` holds the code-sync baseline. A registry entry is `owner/repo: repo-id` or `{repo_id, kind, workflow?, auto_merge?}`; `auto_merge` defaults to `false`.

Path	Holds	Committed
`federation/`	the published snapshots, `index.yaml`, `registry.yaml`	yes
`.kb-auth/` (`users.db`, `session.key`)	accounts and the cookie key; deleting `session.key` logs everyone out	**never**
`.kb-qa/qa.db`	Ticket Q&A	**never**
`.env`, `.kb-work/`, `source/`	the token; the search index and scratch; source PDFs	no

6.3 The MCP tools

The server exposes fourteen tools. Only `kb_question_ask` and `kb_question_answer` write.

Tool	Purpose
`kb_search`	find sections by natural language; returns L2 within a token budget, with citations (`query`, `tags`, `budget`, `kinds`). Each leg is capped at 50 results before fusion, and the tool says when matches were dropped. It also flags when the top two results are the same section published by two repos, when their keyword scores tie exactly, or when both are hybrid-confirmed within the ambiguity ratio
`kb_get_section`	fetch exactly one section at L2 or L3
`kb_context_new`	pin a `kb-context` citation block at the hub's current commit, from confirmed refs
`kb_resolve`	resolve a `kb-context` block (or a ticket containing one): the sections at the pinned version plus their freshness
`kb_ticket_lint`	the Definition-of-Ready gate over a draft ticket
`kb_ticket_check`	the SA grounding gate against the hub's `-code` document
`kb_ticket_stale`	the stale-citation report for one ticket or mission: citations the hub amended or lost since they were pinned, and moved `-code` sections
`kb_mission_lint`	the mission Definition-of-Ready gate
`kb_ticket_fetch`	a ticket a BA published, read from the hub's `<ba-repo>-plan` by id
`kb_question_ask`	**writes.** Ask the BA a question about a published ticket; returns `Q<id>`
`kb_question_list`	list ticket questions with their derived state (`open`, `waiting`, `answered`, or `all`)
`kb_question_answer`	**writes.** Record the BA's answer to an open question
`kb_mission_next`	which story is next, from the plans published on the hub
`kb_doctor_hub`	the hub's side of `kb doctor` for a child, BA or Dev repo (version, published state, kind pin); called by the CLI, not meant for agents

BA skills call every tool except `kb_ticket_fetch`, `kb_question_ask` and `kb_doctor_hub`; Dev skills call `kb_search`, `kb_get_section`, `kb_context_new`, `kb_resolve`, `kb_ticket_lint`, `kb_ticket_check`, `kb_ticket_fetch`, `kb_question_ask` and `kb_question_list`.

6.4 What the server enforces

- Every response carries a CSP (`script-src 'self'`), `X-Frame-Options: DENY`, nosniff, `Referrer-Policy` and `Permissions-Policy`; HSTS only when the request reaches the server over https.
- The `/ui` session cookie is a signed, expiring value keyed with `.kb-auth/session.key`, never with the bearer token (which every repo holds). It carries the username, not the role. The login body is capped at 8 KiB however it is sent.

- Three per-caller rate limiters: `/intake/publish` 30 per minute, `/intake/manifest` and `/intake/status` 120 per minute (shared), and `/ui/login` 5 per minute. A failed `Authorization` attempt shares the login bucket, so the lockout cannot be side-stepped with the header. `STRATA_KB_TRUSTED_PROXIES` governs all of them.
- `GET /intake/status` is public by design (the zero-secret CLI polls it) and, without a token, returns only `state` and `pr_url`, keyed by repo-id and commit: the same internal-network assumption as the MCP. The failure `detail` goes only to a caller that presents that repo's OIDC token.
- Branch protection on the hub plus the OIDC intake are the actual security boundary (2.5): the registry cannot authenticate a git publisher.
- Publish copies `.kb/` **artefacts only**: documents, indexes, manifests and `assets/`. Configuration files and dotfiles are withheld with a `[warn] … (allowlist)` line, so a `config.yaml` someone put a credential in never reaches the hub. `kb doctor` warns when a tracked `config.yaml` carries one.
- `docs/deploy-remote-mcp.md` covers the TLS/HSTS gap and the proxy trust risk (2.3).

6.5 Q&A rules

The store is SQLite at `<hub root>/.kb-qa/qa.db` (in Docker `/data/.kb-qa/qa.db`), created and git-excluded on the first Q&A call. **Never commit `.kb-qa/`**: a new hub's `.gitignore` lists it, and the hub also adds it to `.git/info/exclude` when the web UI starts. Back it up with the rest of the hub volume.

The three tools work only against the hub itself: this server's HTTP endpoint, or the stdio server run inside the hub's own checkout (`--kb <hub>/.kb`). Any other target is refused: a Dev repo's stdio server (even one pointing at the hub's path) or a hub URL read through a local cache clone answers `Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`, because a store there would not be the shared one. BA and Dev repos point MCP at the HTTP server with `kb mcp-setup`.

Any caller the hub authenticates may ask and answer: the bearer token, or a session of any role. `asked_by` and `answered_by` are names the caller types (the skills pass `git config user.name`): tracking, not authentication. Limits: question and answer 4000 characters, context 1000, names 100 (one printable line, no control characters), at most 50 changed sections per answer, and at most 50 open questions per ticket. A question is *open* until answered; an answer that changes the ticket stays *waiting* (shown as "waiting for publish") until the BA's new ticket revision is on the hub, then it is *answered*.

The auto runner of a `ba` or `dev` repo polls `GET /api/questions` with the same bearer token. It is read-only and returns `{"questions": [...]}`. Parameters: `repo`, `ticket`, and `status` (`open` by default, `waiting`, `answered` or `all`; anything else is `400 bad_status`). A hub without a Q&A store answers an empty list. Errors: `400 bad_request` for a malformed `ticket` reference, `503 qa_needs_hub_server` when the hub does not serve its own checkout, `503 hub_unreachable` when the hub cannot be opened, `500 qa_store_error` when the store fails (the detail is in the hub log).

6.6 Exit codes

`0` ok, `1` error or misconfiguration, `2` a stale citation (for `kb doctor`, a stale hub cache) or a CLI usage error. `kb code-sync scan` and `run`: `0` nothing to do, `2` findings, `1` error; `kb code-sync issue`: `0` a usable issue, `2` none, `1` error. Exit `2` for a stale citation comes from `kb resolve`, `kb doctor --context`, `kb ticket lint --fail-on-stale` and `kb mission lint --fail-on-stale`; `kb doctor` also exits `2`, with or without `--context`, when the hub cache itself is stale.